Міністерство освіти і науки України
Центральноукраїнський національний технічний університет
Механіко-технологічний факультет
Кафедра кібербезпеки та програмного забезпечення
Дисципліна: Базові методології та технології програмування

Лабораторна робота №8

Тема: «Реалізація статичних бібліотек модулів лінійних обчислювальних процесів»

Виконав: ст. гр. КН-25
Грицюк Є.В.
Перевірив: викладач
Коваленко А.С.

Кропивницький 2026

ТЕМА: Реалізація статичних бібліотек модулів лінійних обчислювальних процесів

МЕТА: полягає у набутті ґрунтовних вмінь і практичних навичок застосування теоретичних положень методології модульного програмування, реалізації метода функціональної декомпозиції задач, метода модульного (блочного) тестування, представлення мовою програмування C++ даних скалярних типів, арифметичних і логічних операцій, потокового введення й виведення інформації, розроблення програмних модулів та засобів у кросплатформовому середовищі Code::Blocks (GNU GCC Compiler)

Варіант 20

Аналіз і постановка задачі 8.1

Розробити програмний модуль для обчислення значення функції на основі введених користувачем дійсних чисел x, y та z.

Математична модель:

$$S = (z + y + z)^2 - 2\pi\sqrt{\tfrac{1}{3} z}\,.$$

ОДЗ z >= 0

Функція повинна приймати значення змінних від користувача та повертати результат.

Вхідні значення: Три дійсні числа x,y, z (тип даних double).

Вихідні значення: Дійсне число S (тип даних double), що є результатом функції, або спеціальне значення (наприклад, -inf), якщо розрахунок неможливий через порушення області допустимих значень (ОДЗ).

Аналіз вимог, проектування архітектури та детальне проектування програмного модуля.

Функціональні вимоги:

1. Програма має приймати на вхід три значення (x, y, z).
2. Програма зобов'язана валідувати вхідні дані (перевірка z >=0).
3. При коректних даних — виконати обчислення за формулою з використанням константи pi.
4. При некоректних даних (z < 0) — програма не повинна "впасти" з помилкою математичного процесора (NaN), а має коректно перервати обчислення і вивести повідомлення про помилку ОДЗ.

Нефункціональні вимоги:

Модуль має бути реалізований мовою програмування C++ у вигляді окремої функції s_calculation. Ця функція не повинна містити операторів введення/виведення (cin, cout), оскільки вона розробляється як складова частина статичної бібліотеки () для багаторазового використання.

Проектування архітектури

Згідно з принципами модульного програмування, архітектурне рішення для задачі 8.1 реалізовано у вигляді приватної статичної бібліотеки. Фізична структура модуля чітко розділяє інтерфейс та реалізацію, і складається з двох ключових файлів:

1. Заголовковий файл (ModulesHrytsyuk.h): Визначає публічний інтерфейс модуля. Містить декларацію прототипу обчислювальної функції double s_calculation(double x, double y, double z);. Цей файл призначений для підключення (через директиву #include) до зовнішніх компонентів системи, зокрема до модуля модульного тестування (TestDriver) та основного клієнтського застосунку (Hrytsyuk_task).
2. Файл реалізації (main.cpp): містить вихідний код алгоритму обчислення функції s_calculation. Під час етапу збирання (build) цей проект компілюється в об'єктний код і лінкується у результуючий архівний файл статичної бібліотеки з розширенням .a.

Детальне проектування:

Початок. Функція отримує вхідні аргументи x, y, z типу double.

Перевірка ОДЗ: Перевіряється умова підкореневого виразу z < 0.

- Якщо умова виконується (значення z від'ємне), функція генерує виключення (або повертає спеціальне значення NAN) як сигнал про порушення області допустимих значень, і виконання математичних операцій переривається.
- Якщо ні (значення z >= 0), відбувається перехід до наступного кроку.

Декомпозиція обчислень: Для уникнення помилок у написанні коду, оптимізації та підвищення його читабельності, математичний вираз розбивається на дві логічні складові (терми):

- Обчислення першої частини (квадрат суми): term1 = pow(x + y + z, 2.0) (для оптимізації швидкодії в коді може бути замінено на (x + y + z) * (x + y + z)).
- Обчислення другої частини (добуток константи на корінь): term2 = 2.0 * M_PI * sqrt(z / 3.0).

Формування результату: Обчислюється кінцеве значення за формулою S = term1 - term2.

Кінець. Функція повертає обчислене значення S типу double у точку виклику.

Artifact: Test Suite

Date: DATE \@ "M/d/yyyy"
10/22/2024

Назва тестового набору Test Suite Description	TS_lab8
Назва проекта / ПЗ Name of Project / Software	Грицюк-task_8_1.exe
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	1. Запустити застосунок TestDriver.exe 2. Передати у функцію s_calculation нульові значення (границя ОДЗ): x = 0.0, y = 0.0, z = 0.0	Функція повертає значення S: 0.0000 Вікно застосунку виводить: [PASS]	passed
TC-02	1. Запустити застосунок TestDriver.exe 2. Передати у функцію s_calculation типові додатні аргументи: x = 1.0, y = 1.0, z = 3.0	Функція повертає значення S: 18.7168 Вікно застосунку виводить: [PASS]	passed
TC-03	1. Запустити застосунок TestDriver.exe 2. Передати у функцію s_calculation дробові значення та від'ємне x: x = -2.5, y = 4.5, z = 1.2	Функція повертає значення S: 6.2662 Вікно застосунку виводить: [PASS]	passed
TC-04	1. Запустити застосунок TestDriver.exe 2. Передати у функцію s_calculation великі числа різного знаку: x = 10.0, y = -20.0, z = 12.0	Функція повертає значення S: -8.5664 Вікно застосунку виводить: [PASS]	passed
TC-05	1. Запустити застосунок TestDriver.exe 2. Передати у функцію s_calculation значення поза ОДЗ (від'ємний підкореневий вираз): x = 5.0, y = 5.0, z = -1.0	Функція генерує виключення invalid_argument (або помилка). Вікно застосунку виводить: [PASS] ... (ОДЗ успішно відхилено)	passed

Лістинг задачі 8.1

```
#include <cmath>

#include <stdexcept>
```

```
constexpr double PI = 3.14159265358979323846;


double s_calculation(double x, double y, double z) {
    if (z < 0.0) {
        throw std::invalid_argument("z не може бути від'ємним");
    }
    double term1 = pow(x + y + z, 2.0);
    double term2 = 2.0 * PI * sqrt(z / 3.0);
    return term1 - term2;
}
```

Заголовковий файл

```
#ifndef MODULES_HRYTSIUK_H
#define MODULES_HRYTSIUK_H


double s_calculation(double x, double y, double z);


#endif
```

Тестовий драйвер

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include "ModulesHrytsiuk.h"


using namespace std;
```

```
void runTestCase(int tc_id, double x, double y, double z, double expected) {
    double actual;

    try {
        actual = s_calculation(x, y, z);
    } catch (...) {
        actual = NAN;
    }

    cout << "TC-" << tc_id << " | x=" << x << ", y=" << y << ", z=" << z << endl;
    cout << "Очікувано: " << expected << " | Фактично: " << actual << " | Статус: ";

    if (isnan(expected) && isnan(actual)) {
        cout << "passed" << endl;
    }
    else if (abs(actual - expected) < 0.01) {
        cout << "passed" << endl;
    }
    else {
        cout << "failed" << endl;
    }
}

int main() {
    setlocale(LC_ALL, "");

    cout << "Протокол модульного тестування функції s_calculation" << endl;
```

```
    cout << "----------------------------------------------------" << endl;


    runTestCase(1, 0.0, 0.0, 0.0, 0.0000);

    runTestCase(2, 1.0, 1.0, 3.0, 18.7168);

    runTestCase(3, -2.5, 4.5, 1.2, 6.2662);

    runTestCase(4, 10.0, -20.0, 12.0, -8.5664);

    runTestCase(5, 5.0, 5.0, -1.0, NAN);


    cout << endl;

    system("pause");

    return 0;

}
```

Результати

```
Протокол модульного тестування функції s_calculation
----------------------------------------------------
TC-1 | x=0, y=0, z=0
Очікувано: 0 | Фактично: 0 | Статус: passed
TC-2 | x=1, y=1, z=3
Очікувано: 18.7168 | Фактично: 18.7168 | Статус: passed
TC-3 | x=-2.5, y=4.5, z=1.2
Очікувано: 6.2662 | Фактично: 6.26616 | Статус: passed
TC-4 | x=10, y=-20, z=12
Очікувано: -8.5664 | Фактично: -8.56637 | Статус: passed
TC-5 | x=5, y=5, z=-1
Очікувано: nan | Фактично: nan | Статус: passed
```

Аналіз і постановка задачі 8.2

Вхідні дані: Два символи a і b (char) для перевірки логічного виразу та три дійсні числа x, y, z (double) для обчислення математичної функції.

Вихідні дані: Інформація про розробника, результат логічного виразу, значення S.

Функціональні вимоги:

- Програма повинна послідовно запитувати та зчитувати значення x, y, z, a, b.
- Реалізувати функцію для генерації рядка авторства.
- Реалізувати функцію для перевірки умови a + 1 < b. Звернути увагу на не явне зведення типів.
- Реалізувати функцію для форматування результатів. Оскільки x, y, z є дійсними числами, для їх виведення у шістнадцятковому форматі необхідно застосувати специфікатор std::hexfloat.
- Підключити та використати модуль ModulesHrytsiuk.

Нефункціональні вимоги:

- Дотримання вимоги щодо використання оператора вставки << у потоці виводу для результатів розроблених функцій.

Проектування архітектури:

Застосунок будується як монолітний консольний клієнт, що залежить від зовнішньої статичної бібліотеки libModulesHrytsiuk.a. У файлі main.cpp будуть визначені три допоміжні функції, які відповідають за генерацію необхідного контенту для виводу. Для того, щоб функція могла повертати складний відформатований текст, її типом повернення буде string. Всередині неї використовуватиметься об'єкт sstringstream для конкатенації чисел та тексту.

Детальне проектування:

Початок. Запуск програми, ініціалізація стандартних потоків введення-виведення, налаштування кодування для коректного відображення кирилиці.
Введення даних:

- Програма виводить підказку для користувача.
- Зчитує з потоку cin послідовність даних: дійсні значення змінних x, y, z (тип double) та символьні значення a, b (тип char).

Виведення авторства (Підзадача 8.2.1):

- Викликається розроблена функція get_developer_info(), яка повертає рядок із символом копірайту та прізвищем.

- Повернене значення включається у вихідний потік cout за допомогою оператора вставки <<.

Виведення логічного результату (Підзадача 8.2.2):

- Викликається розроблена функція check_logical_expr(a, b).
- Функція обчислює значення виразу a + 1 < b (де символи неявно приводяться до їхніх ASCII-кодів для виконання арифметичної операції) і повертає ціле число або булеве значення.
- Результат включається у потік cout. Застосовується маніпулятор noboolalpha (або стандартна поведінка C++), щоб гарантувати числовий формат виводу (1 або 0).

Виведення систем числення та обчислень (Підзадача 8.2.3):

- Викликається розроблена функція format_calculations(x, y, z).
- Всередині функції: Здійснюється виклик функції s_calculation(x, y, z) з підключеного заголовкового файлу ModulesHrytsiuk.h.
- Блок try-catch перехоплює можливе виключення (порушення ОДЗ, z < 0) і формує відповідне текстове повідомлення замість результату.
- За допомогою об'єкта std::stringstream формується єдиний текстовий рядок, який містить: значення x, y, z у стандартному десятковому форматі; значення x, y, z у шістнадцятковому форматі (із застосуванням маніпулятора hexfloat); обчислене значення S.
- Сформований рядок повертається з функції та включається у потік cout.

Кінець

Лістинг 8.2

```
#include <iostream>

#include <iomanip>

#include <string>

#include <sstream>

#include <cmath>

#include <Windows.h>
```

```
#include "ModulesHrytsiuk.h"

using namespace std;

string get_developer_info() {
    return "© Євгеній Грицюк";
}

int check_logical_expr(char a, char b) {
    return (a + 1 < b) ? 1 : 0;
}

string format_calculations(double x, double y, double z) {
    stringstream ss;

    ss << "Значення x, y, z (dec): " << x << ", " << y << ", " << z << "\n";
    ss << "Значення x, y, z (hex): " << hexfloat << x << ", " << y << ", " <<
z << defaultfloat << "\n";
    ss << "Значення S            : ";

    try {
        double s = s_calculation(x, y, z);
        ss << s;
    } catch (...) {
        ss << "NAN (Порушення ОДЗ)";
    }

    return ss.str();
```

```
}

int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);

    double x, y, z;
    char a, b;

    cout << "--- ВВЕДЕННЯ ДАНИХ ---" << endl;
    cout << "Введіть дійсні числа x, y, z через пробіл: ";
    if (!(cin >> x >> y >> z)) {
        cerr << "Помилка вводу чисел!" << endl;
        return 1;
    }

    cout << "Введіть два символи a та b через пробіл: ";
    if (!(cin >> a >> b)) {
        cerr << "Помилка вводу символів!" << endl;
        return 1;
    }

    cout << "\n--- РЕЗУЛЬТАТИ (Вивід через cout) ---" << endl;

    cout << get_developer_info() << endl;
    cout << "Результат a + 1 < b   : " << check_logical_expr(a, b) << endl;
    cout << format_calculations(x, y, z) << endl;
```

```
    cout << "\n";

    system("pause");

    return 0;

}
```

```
--- ВВЕДЕННЯ ДАНИХ ---
Введіть дійсні числа x, y, z через пробіл: 2 5.2 3.1
Введіть два символи a та b через пробіл: p o

--- РЕЗУЛЬТАТИ (Вивід через cout) ---
© Євгеній Грицюк
Результат a + 1 < b   : 0
Значення x, y, z (dec): 2, 5.2, 3.1
Значення x, y, z (hex): 0x1p+1, 0x1.4ccccccccccccdp+2, 0x1.8ccccccccccccdp+1
Значення S            : 99.703
```

Аргументи

1. Розуміння фундаментальної різниці між інтерфейсом (.h) та реалізацією (.cpp).
2. Опанування принципу приховування даних (інкапсуляції) на рівні файлової системи.
3. Отримання практичного досвіду створення приватних статичних бібліотек (.a).
4. Чітке усвідомлення етапів збірки програми: препроцесинг, компіляція, лінкування.
5. Набуття навички налаштування Search directories для компілятора в IDE.
6. Набуття навички налаштування Linker settings для компонувальника.
7. Розуміння природи помилки undefined reference та вміння її виправляти.
8. Розуміння помилки multiple definition та принципів One Definition Rule (ODR).

9. Відмова від небезпечних студентських милиць на кшталт #include "file.cpp".
10. Розуміння механіки Include Guards (#ifndef) для захисту від подвійного підключення.
11. Знайомство з концепцією модульного тестування (Unit Testing).
12. Практика написання власного тестового драйвера з нуля.
13. Розуміння архітектурної різниці між модульним та системним тестуванням.
14. Складання професійної документації: тест-сьюта (Test Suite).
15. Практика тестування граничних умов (Boundary Testing) та перевірки ОДЗ.
16. Опанування механізму перехоплення виключень (блок try-catch).
17. Робота зі специфічними станами пам'яті: значеннями NaN (Not-a-Number) та Infinity.
18. Розуміння специфіки порівняння дійсних чисел через похибку (епсилон).
19. Використання стандартної математичної бібліотеки <cmath>.
20. Поглиблене розуміння потоків введення-виведення (iostream).
21. Практика безпечного зчитування даних і валідації вводу (if (!(cin >> x))).
22. Робота з форматуванням тексту в консолі за допомогою бібліотеки <iomanip>.
23. Використання сучасного маніпулятора hexfloat для шістнадцяткового представлення дійсних чисел.
24. Опанування потоків у пам'яті (std::stringstream) для складної конкатенації тексту.
25. Засвоєння патерну розділення логіки обчислень та логіки виводу (функція повертає рядок, а не друкує його).

26. Усвідомлення принципів неявного приведення типів (наприклад, арифметичні операції над char через ASCII-коди).
27. Практика діагностики та виправлення проблем із кодуванням символів (UTF-8 проти CP1251).
28. Використання Windows API (<Windows.h>) для налаштування локалі консолі.
29. Розуміння проблеми невидимих символів BOM (Byte Order Mark) та помилок препроцесора.
30. Вирішення міжплатформених конфліктів із закінченням рядків (CRLF проти LF).
31. Налаштування IDE (Code::Blocks) для роботи за стандартами Unix та Git.
32. Закріплення консольних навичок роботи з системою контролю версій Git.
33. Розуміння важливості .gitignore для ізоляції вихідного коду від скомпільованих бінарників (obj, bin).
34. Написання осмислених, професійних повідомлень для комітів (Conventional Commits).
35. Розуміння механізму core.autocrlf у Git при роботі на ОС Windows.
36. Розвиток навички читання технічного завдання (ТЗ) зі стовідсотковою увагою до пунктуації.
37. Вміння чітко розмежовувати вимоги до вхідних та вихідних даних.
38. Практика створення детального алгоритму до початку написання коду.
39. Набуття досвіду документування результатів проектування (алгоритми, кроки тестування).
40. Звикання до суворої інженерної дисципліни та дотримання ТЗ без самодіяльності.

41. Розуміння внутрішньої механіки C++ без використання абстракцій високорівневих мов.
42. Усвідомлення переваг статичних бібліотек для зменшення часу перекомпіляції великих проектів.
43. Здатність передавати замовнику готовий виконуваний код без розкриття вихідників.
44. Підготовка бази для розуміння того, як підключаються реальні комерційні фреймворки та SDK.
45. Виховання звички шукати першопричину багів (root cause), а не застосовувати випадкові виправлення.
46. Усвідомлення того, що статус "воно працює" нічого не вартий, якщо порушена архітектура проекту.
47. Доведення навичок базового синтаксису C++ до автоматизму.
48. Отримання досвіду вирішення багів інтеграції (коли два незалежні проекти мають запрацювати разом).
49. Зменшення страху перед консольними помилками компілятора та лінкера.
50. Формування міцного структурного фундаменту, необхідного для подальшого переходу до Об'єктно-орієнтованого програмування (ООП).

Висновок: У цій лабораторній роботі здійснено перехід до розуміння базової індустріальної архітектури програмного забезпечення. На практиці закріплено повний цикл розробки: проектування модульної структури, створення та лінкування статичних бібліотек, написання тестових драйверів для системного тестування, а також вирішення реальних інтеграційних проблем із кодуваннями та системами контролю версій.

Здобуті навички формують міцний інженерний фундамент, необхідний для подальшого створення масштабованого коду у комерційній розробці.